MaCVi@WACV 2027 EURS Challenge
Embedded Underwater Restoration and Segmentation
Factsheet

[Author 1], [Author 2], [Author 3]
[Affiliation, City, Country]

Instructions: how to submit the factsheet

This factsheet describes the contribution of each participating team in the EURS challenge at MaCVi@WACV 2027. Please address all aspects listed below as completely as you can. The factsheet, the code and the achieved performance on the testing-phase data are used to verify the reported results and to decide on the final EURS ranking.

1. **Reproducibility is a must.** Only reproduced testing-phase results are ranked. The reproduction of your submission must match the testing-phase submission you report here, within normal run-to-run variation.
2. **Detailed descriptions are much appreciated:** a brief description of all models and experiments you tested, ablations, visualisations, things that did not work, external datasets, pre-trained models, ensembles, and so on. Disclose all external data and pre-trained models you used, in the same way as required on the leaderboard.
3. **Fill in one factsheet per team.** If your team took part in both tracks, describe both tracks in the same factsheet. Every team completes Sections 1 and 2 and the section of each track it entered (Section 3 for Track 1, Section 4 for Track 2); teams that entered only one track leave the other track section out.

What to send. All factsheets must be sent by e-mail to the EURS track leads and chairs listed below, with all your team members in cc:

- **A compiled PDF** of the factsheet.
- **A** `.zip` with the LaTeX source files of the factsheet (`main.tex`, `ref.bib`, `images/`, including all figures).

The factsheet is separate from the code and model weights you hand in for reproduction: provide the code and model-weight link in Section 1 (Team details) and the exact reproduction commands in Section 6.

Code and weights: use our repository template. Hand in your code in the format and structure of the official EURS reproduction template at `https://github.com/CeviKle/EURS-MaCVi2027-repro-template` (click *Use this template*; one repository per track entered). Follow its README for the required files, the weights rules and the check script (`repro_check.py`) to run before you hand in. The repository link goes into Section 1. **Track 2:** the repository also contains `export_onnx.py`, the weights and a copy of the ONNX file you uploaded (`onnx/submitted_model.onnx`); we regenerate your ONNX in a sandbox, compare it with your leaderboard file, and evaluate it again on the Jetson AGX Orin, for every team.

E-mail guide.

To: nikhil.akalwadi@kletech.ac.in;
sampadamalagi12@gmail.com;
sneha.varur@kletech.ac.in;
chaitra.desai@manipal.edu

Organizer	Email
Nikhil Akalwadi	nikhil.akalwadi@kletech.ac.in
Sampada Malagi	sampadamalagi12@gmail.com
Sneha Varur	sneha.varur@kletech.ac.in
Chaitra Desai	chaitra.desai@manipal.edu

Cc: all your team members

Subject: MaCVi@WACV 2027 EURS Challenge - TEAM_NAME

The body of the e-mail should contain:

a) the challenge name (MaCVi@WACV 2027 EURS Challenge),

b) your team name (exactly as registered on macvi.org),

c) the team leader's name and e-mail address,

d) the rest of the team members,

e) the macvi.org user IDs of all team members, including the team leader,

f) the ID and the submitting user ID of your best-scoring testing-phase submission, one per track,

g) the link to your reproduction repository (created from the official template) and to your trained weights,

h) the factsheet PDF and the LaTeX source `.zip` attached.

Deadline. October 28, 2026, 23:59 AoE (Anywhere on Earth). Factsheets received after the deadline will not be considered for evaluation.

---

1 Team details

- **Team name (as on macvi.org):** [team name]
- **Team leader name:** [name]
- **Team leader institution and email:**
  [institution, city, country]
  Email: [leader e-mail]
- **Rest of the team members (name, macvi.org user ID):**
  - [member 1], user ID [ID]
  - [member 2], user ID [ID]
  - [. . .]
- **Team website URL (if any):** [URL]
- **Affiliations:** [affiliations]
- **Team leader's user ID on the submission server (macvi.org):** [ID]
- **User IDs on the submission server of *all* team members** (leader included; the IDs of every account that submitted for the team): [ID, ID, . . .]
- **Track(s) entered:** [Track 1 / Track 2 / both]
- **Best-scoring testing-phase submission per track (the one reported for reproduction):**
  - Track 1 (testing-phase submission used for the final ranking): submission ID [ID], submitted by user ID [ID], composite score [score]
  - Track 2 (testing-phase submission used for the final ranking; Jetson AGX Orin, ONNX): submission ID [ID], submitted by user ID [ID], composite score [score]
- **Link to the reproduction repository (created from the official EURS template, see the instructions on page 1):** [URL of your repository, one per track entered; the weights of the reported submission must be in it or behind the link given in `weights/weights_link.yaml`]
- **Artifact traceability:** code revision / commit [hash]; final checkpoint or ONNX file name [name]; SHA-256 of the final artifact [hash]; link access status and expiry date (if any) [details]

2 Global method description

[General method description: the architecture, how restoration and segmentation share (or do not share) features, what is new compared with published work. Add equations if useful. Give credit to all works your solution builds on, for example U-Net-style encoders [1].]

- **Which pre-trained or external methods / models have been used (for any stage, if any):** [list them with citations, or state "None"]
- **Which additional data has been used in addition to the provided EURS training data (at any stage, if any):** [dataset names, citations, number of images, how they were used (pre-processing, filtering); or "None"]
- **Training description:** [framework, optimizer, learning rate schedule, batch size, number of epochs, input/patch size, augmentations, loss functions and weights, hardware, training time]
- **Testing description:** [input resolution at inference, any resizing to the scoring size (exactly 1024 px on the longest side, aspect ratio kept), test-time augmentation or ensembling (if any), hardware]

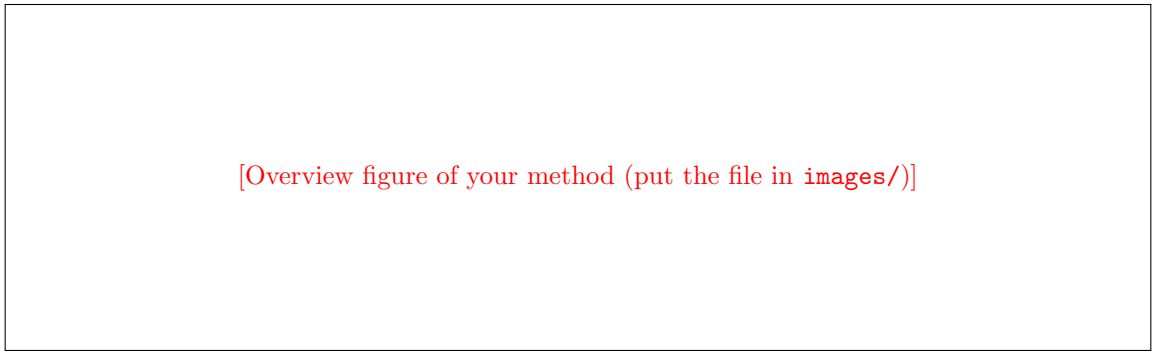


Figure 1: [Overview of the proposed method. Explain the main blocks and the information flow for both outputs (restored image and segmentation mask).]

- **How the training and development scenes were used:** [list the exact scene IDs used for training, for validation and for model selection; state whether the 6 development scenes or the held-out scenes were used in any of these]
  Compliance with the EURS rules (challenge page and hand-in announcement), including disclosure of all additional training data: [Yes / No; explanation]
- **Were the released testing-phase images (inputs) used in any way for training, tuning or model selection (including pseudo-labelling or self-supervised adaptation)?** [Yes / No; if Yes, explain exactly how]
- **Novelty degree of the solution and whether it has been previously published:** [text]
- **Other methods and baselines tested (even if results were not top competitive):** [text]

3 Track 1: restoration and segmentation outputs

Skip this section if you did not take part in Track 1.

- **Submission ID reported for the final ranking:** [ID]
- **Model checkpoint used to produce this submission:** [file name / epoch / commit]
- **How the restored images and the five-class masks are produced:** [one model or two; how each output is obtained from one input image only]
- **Post-processing (if any):** [text, or "None"]
- **Inference time per image:** [ms, on which GPU, at which resolution]

Model	Input size	Epochs	Extra data	Ensemble / TTA	# Params. (M)	GPU
[name]	[H×W]	[N]	[Yes/No]	[Yes/No]	[M]	[GPU]

Table 1: Training statistics and computational metrics for Track 1.

4 Track 2: ONNX model on the Jetson AGX Orin

Skip this section if you did not take part in Track 2.

- **Submission ID reported for the final ranking:** [ID]

- **How the ONNX file is produced from the trained weights:** [export script, opset, any simplification or graph changes, quantization (if any)]
- **Copy of the submitted ONNX file in your repository:** path [`onnx/submitted_model.onnx`]; SHA-256 [hash]; this is the exact file you uploaded to the leaderboard for the submission above
- **Software used to produce the submitted ONNX file:** [torch / onnx / exporter versions, opset (a different version only changes the file header, which is fine)]
- **Does `python repro_check.py . --export-onnx` regenerate the submitted file from your weights?** [Yes (byte-identical) / Yes (same graph and weights, header differs) / No; explanation]
- **Static input shape** $(1, 3, H, W)$**:** [H]×[W]; **outputs:** `restored_image`, `seg_logits`
- **Complexity of the submitted model:** [GFLOPs] (limit 12) and [M parameters] (limit 1.0 M)
- **Efficiency measures reported by the pipeline for this submission:** throughput [FPS], energy per frame [value]
- **Is the model identical to the one used in Track 1?** [Yes / No; describe the difference]
- **Efficiency strategies (pruning, distillation, depthwise convolutions, . . .):** [text]

Model	Input (H, W)	Precision	# Params. (M)	GFLOPs	Training time	Epochs	Extra data
[name]	[H×W]	[FP32/FP16]	[M]	[G]	[h]	[N]	[Yes/No]

Table 2: Training statistics and computational metrics for Track 2.

5 Implementation and environment

Give the optimizer, learning rate, epochs, loss functions and training data once, in Section 2; do not repeat them here.

- **Programming language and framework (with versions):** [e.g. Python, PyTorch]
- **Export and runtime versions (ONNX, opset, TensorRT-related tools, if used):** [versions]
- **Operating system and hardware used for training and for the reproduction run:** [OS, GPU/CPU, memory]
- **External code and dependencies (cite all external code):** [repositories, licences, link to a dependency lock file such as `requirements.txt` or `environment.yml`]
- **Efficiency optimization strategies (if any):** [text]

6 Reproduction procedure

- **Installation commands:** [exact commands, in order]
- **Track 2: command that regenerates the ONNX file (normally `python repro_check.py . --export-onnx`):** [command, or "not applicable (Track 1 only)"]
- **Inference commands and entry-point script:** [exact commands, or point to the section of the README in your repository]
- **Checkpoint file name and SHA-256 hash:** [name, hash]

- **Expected input and output structure:** [e.g. `restored/` and `seg_masks/` with one file per input image]
- **Fixed seeds and any nondeterministic operations (`model.eval()`, no test-time randomness):** [text]
- **Expected reproduced result:** [the score you expect from the reproduction; it should match your reported testing-phase result within normal run-to-run variation]
- **Approximate run time of the full reproduction:** [minutes, GPU]

7 Other details

- **Planned submission of a solution description paper to the MaCVi@WACV 2027 workshop:** [Yes / No]
- **General comments and impressions of the EURS challenge (we appreciate your feedback to improve future editions):** [text]
- **What do you expect from a new challenge in underwater image restoration and segmentation?** [text]
- **Known limitations and failure cases of the method:** [text]

Declaration

I confirm that all external datasets, pretrained models, ensembles and post-processing used for the reported submission(s) are disclosed in this factsheet; that the submitted code and weights correspond to the named testing-phase submission(s); and that the method complies with the official EURS rules.
Team leader: [name] Date: [date]

References

[1] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.